



Fondamenti di Ingegneria del Software (Appunti per l'Esame)

L'Ingegneria del Software non è "scrivere codice", ma l'applicazione di un **approccio sistematico, quantificabile e disciplinato** allo sviluppo, al funzionamento e alla manutenzione del software.



1. Il Ciclo di Vita del Software

Le fasi attraverso cui passa un prodotto software. Spesso non sono rigidamente sequenziali.

1. **Studio di Fattibilità:** Vale la pena farlo? Si valutano costi, benefici e risorse.
2. **Analisi dei Requisiti:** Cosa deve fare il sistema? Si formalizzano le specifiche.
 - *Funzionali:* Cosa il sistema fa (relazione input-output).
 - *Non Funzionali:* Come il sistema si comporta (qualità generali come sicurezza, velocità).
3. **Progettazione:** Come lo costruiamo? Si definisce l'architettura ad alto livello e si divide il sistema in moduli (interfacce definite, dettagli interni nascosti).
4. **Codifica e Test:** Scrittura del codice (implementazione) e test dei singoli moduli isolati.
5. **Integrazione:** Assemblaggio dei moduli e test del sistema nel suo complesso.
6. **Consegna e Manutenzione:** Rilascio al cliente (Stakeholder). **È la fase più lunga e costosa** dell'intero ciclo di vita (bug fixing, aggiornamenti).



2. Le Qualità del Software (Il Metro di Giudizio)

Si dividono in *Esterne* (percepite dall'utente) e *Interne* (percepite dagli sviluppatori).

- **Correttezza:** Il software rispetta *assolutamente* i requisiti specificati? (Sì/No).
- **Affidabilità:** Probabilità che il software continui a funzionare correttamente nel tempo senza guasti.
- **Robustezza:** Capacità del software di comportarsi in modo "ragionevole" anche in situazioni impreviste (es. input errati, caduta di rete) non descritte nelle specifiche.
- **Prestazioni (Efficienza):** Uso ottimale delle risorse hardware (CPU, RAM). Valutabile con complessità asintotica o misurazioni dirette.
- **Manutenibilità ed Evolvibilità:** Facilità con cui è possibile trovare/correggere bug (manutenzione) e aggiungere nuove funzionalità per adattarsi a nuovi scenari (evoluzione).
- **Riusabilità e Interoperabilità:** Capacità di usare pezzi di codice per altri progetti (es. Design Pattern, librerie) e capacità di dialogare con altri sistemi software tramite interfacce standard.



3. Principi Fondamentali (Come si ottiene la qualità)

- **Rigorosità e Formalità:** La precisione ingegneristica nella documentazione e nei processi, fondamentale per rispettare tempi e costi.

- **Separazione degli Interessi (Separation of Concerns):** Affrontare un problema complesso dividendolo in problemi più piccoli (per tempo, per qualità, per responsabilità).
- **Modularità (DOMANDA DA ESAME 🔥):** Dividere il sistema in moduli. Per un'ottima modularità serve:
 - **Alta Coesione (High Cohesion):** I componenti *all'interno* di un modulo devono essere strettamente legati e focalizzati su un singolo scopo (es. Principio di Singola Responsabilità).
 - **Basso Accoppiamento (Low Coupling):** I moduli *tra di loro* devono essere il più indipendenti possibile. Se modifico il Modulo A, il Modulo B non deve rompersi.
- **Astrazione e Incrementabilità:** Ignorare i dettagli di basso livello (Black Box) per capire il quadro generale, e procedere per rilasci e approssimazioni successive.



4. Modelli e Metodologie di Sviluppo

Come i team si organizzano per lavorare. Si è passati dall'approccio grezzo "Code & Fix" a modelli strutturati.

Modelli Tradizionali / Strutturati

- **Modello a Cascata (Waterfall):** Fasi rigorosamente sequenziali. Non si passa alla fase successiva se non è finita la precedente. **Pro:** Facile da gestire. **Contro:** Rigidissimo, non gestisce i cambiamenti, l'utente vede il prodotto solo alla fine.
- **Modello a Spirale:** Basato sull'analisi dei *rischi*. Si procede a cicli (spirali) valutando i costi e i rischi prima di ogni incremento.
- **RUP (Rational Unified Process):** Metodologia iterativa basata su UML e guidata dai Casi d'Uso (Use Cases). Divisa in Concezione, Elaborazione, Costruzione e Transizione.

Metodologie Agili (XP, Scrum)

Nate per abbattere la rigidità del Waterfall e rispondere ai mercati che cambiano in fretta. Prediligono il *codice funzionante* alla documentazione eccessiva, e la *collaborazione col cliente* alla negoziazione dei contratti.

- **Extreme Programming (XP):** Estremizza le pratiche di sviluppo. Rilasci frequentissimi, *Test-First Development* (scrivi prima il test, poi il codice), e *Pair Programming* (due programmatori su un solo PC).
- **Scrum:** Metodologia iterativa famosissima. I requisiti sono una to-do list dinamica (*Product Backlog*). Il lavoro è diviso in iterazioni fisse di 2-4 settimane chiamate **Sprint**. Il team è autogestito e facilitato da una figura chiamata *Scrum Master* (che non è il capo, ma un facilitatore).